SE-2622

Adiyat, Artur, Erdaulet

OUR PROJECT THEME: ONLINE ORDER SYSTEM FOR A CAFÉ

Project Stage 1

Part A — Problem definition and team charter

A1

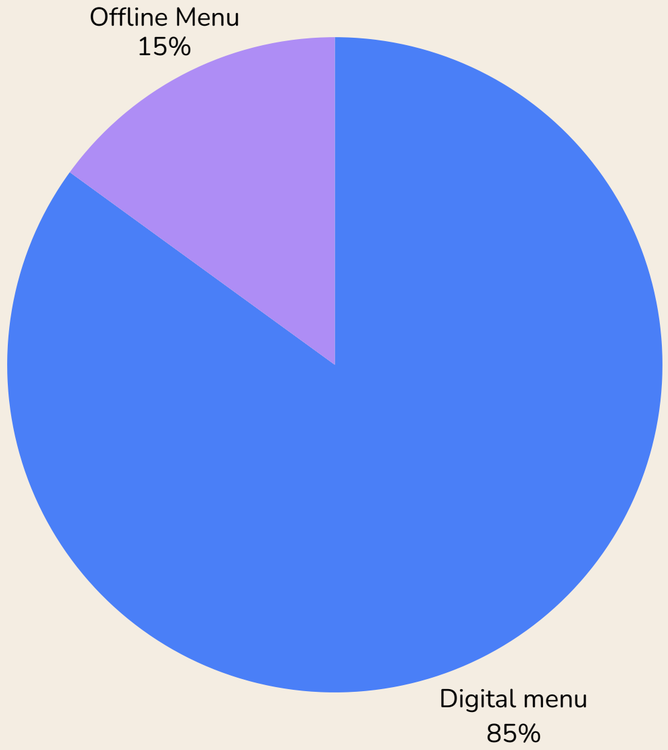
PROBLEM STATEMENT

Many small cafés receive orders directly at the counter, by phone or through messaging applications. During busy hours customers may need to wait in a queue before they can order. Staff also need to manually write down or remember orders, which can sometimes cause mistakes, incorrect items or delays.

Customers may also have difficulty checking the full menu, prices and available products before they arrive at the café.

Our project is an online order system for a small café. Customers will be able to open the website, view the menu, select products, add them to a cart and send an order before arriving at the café. The café staff will receive the order through an administration panel and update its status.

The system can reduce waiting time, make ordering easier and help staff manage orders in one place. It can also reduce communication errors because the customer selects products directly from the digital menu.



More and more, guests are beginning their journey online. 85% said they look at the menu digitally first

Source:
https://www.fsrmagazine.com/feature/what-do-restaurant-customers-want-in-2024-savings-convenience-and-connection

A2. USERS

Customer

Wants to view the café menu

See prices

Select products

Add products to the cart

Place an order and check the order status

Café Staff / Administrator

Wants to view new orders

Update order status

Manage menu items

Change prices and mark unavailable products.

A2. SCOPE

In scope	Out of scope
View café menu	Employee payroll
Product categories	Full accounting system
View prices	Supplier management
Add products to cart	Restaurant table reservation
Remove products from cart	Delivery courier tracking
Place an order	Full CRM
View order status	AI food recommendations
Admin order panel	Government integrations
Manage menu	Inventory accounting system
Mark products unavailable	Loyalty program in first version

A3. GOAL AND SUCCESS CRITERIA

The goal of the project is to create a simple online ordering system that allows café customers to select products and place an order without waiting in a physical queue.

01. A customer can place an order in less than 2 minutes.

02.An unavailable product cannot be added to a new order.

03. The main ordering interface works correctly on a mobile screen with a width of 360 pixels.

A4. TEAM CHARTER

Team members

Artur

UI Designer

UI/UX Design, wireframes, usability analysis &

Development: Refining methodologies and developing core frameworks.

Erdaulet

Researcher / Presenter

Evaluation & Future Scaling: Assessing impact and planning for sustained growth and evolution.

Adiyat

Analyst / Developer

Requirements, architecture, data flow

PART B — REQUIREMENTS

FUNCTIONAL REQUIREMENTS

ID	Requirement	Type	Priority	How it will be
R1	User can view the café menu	Functional	Must	Open menu page
R2	User can view product prices	Functional	Must	Open product card
R3	User can add a product to the cart	Functional	Must	Add test product
R4	User can remove a product from the cart	Functional	Must	Remove test product
R5	User can change product quantity	Functional	Should	Change quantity in cart
R6	User can place an order	Functional	Must	Create test order
R7	User can view order status	Functional	Should	Open test order
R8	Administrator can view incoming orders	Functional	Must	Open admin panel
R9	Administrator can change order status	Functional	Must	Change New to Preparing
R10	Administrator can add or edit menu products	Functional	Should	Edit test product
R11	Administrator	Functional	Must	Disable test

NON-FUNCTIONAL REQUIREMENTS

Performance

- R12: The menu and cart pages should load in less than 2 seconds.

Security

- R13: Communication between the customer and server must use HTTPS.

Usability

- R14: A customer should be able to place an order in no more than five main steps.

Availability

- R15: The service should be available at least 99% of the time.

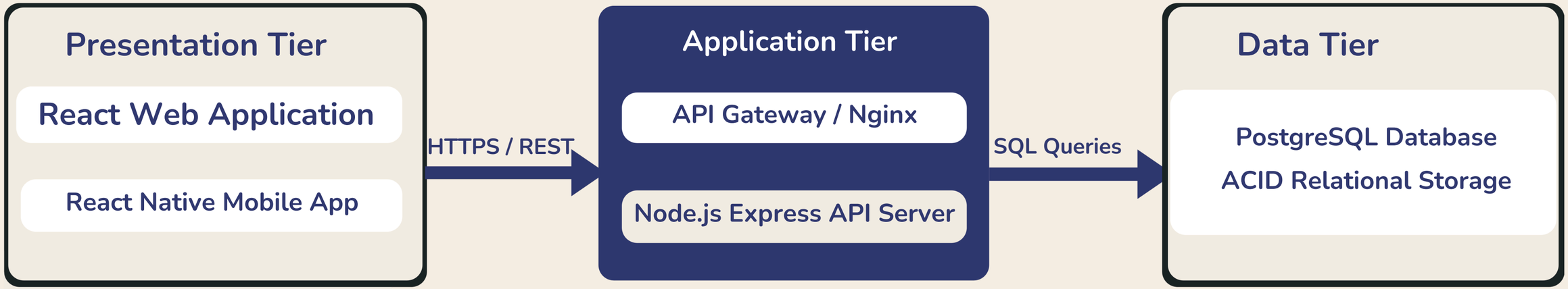
THREE MOST IMPORTANT REQUIREMENTS

The three most important requirements are viewing the menu, adding products to the cart and placing an order. Viewing the menu is important because customers need to see available products and prices before making a decision. The cart allows customers to check selected products, quantities and total price before submitting the order. Placing an order is the main function of the system because without it the service cannot solve the problem of physical queues. These three requirements create the basic ordering process. Other features such as order history, advanced filters or interface customization can be added later because they are not required for the first working version.

PART C1 — ARCHITECTURE & DATABASE DESIGN

3-Tier System Model & Core Entity Schema

1. 3-TIER SYSTEM ARCHITECTURE DIAGRAM



2. RELATIONAL DATABASE SCHEMA DESIGN

1. Categories

id (PK, UUID)
name (VARCHAR 100)
description (TEXT)
created_at (TIMESTAMP)

2. Products

id (PK, UUID)
category_id (FK -> Categories.id)
name (VARCHAR 200)
price (DECIMAL 10,2)
stock_quantity (INT)

3. Orders

id (PK, UUID)
user_id (FK -> Users.id)
total_amount (DECIMAL 10,2)
status (VARCHAR: pending/paid)

4. Order_Items

id (PK, UUID)
order_id (FK -> Orders.id)
product_id (FK -> Products.id)
quantity (INT), unit_price (DECIMAL)

Architectural Justification: The 3-tier architecture decouples the frontend client interface from backend processing and database persistence. This prevents direct database access from clients, ensures modular code maintainability, and allows independent scaling of backend services during peak operational hours.

Part C2 — Technology Stack Selection

ComprehensiveTechnicaUustification&Framework Matrix

TECHNOLOGY STACK BREAKDOWN

Component Layer	Chosen Technology	Technical Justification
Client Platform	React.js & React Native	Provides high interface responsiveness, cross-platform UI adaptability for web and mobile, and a reusable component model.
Web Languages	TypeScript & JavaScript	TypeScript enforces strict compile-time type safety, minimizing runtime exceptions and accelerating long-term maintenance.
Server Technology	Node.js (Express.js)	Event-driven, non-blocking asynchronous I/O architecture ideal for handling concurrent API traffic with minimal resource overhead.
Database System	PostgreSQL	Enterprise-grade relational database guaranteeing full ACID compliance, high query reliability, and native JSON storage support.
Cloud Platform	AWS (Amazon Web Services)	Industry-standard cloud ecosystem providing auto-scaling compute (EC2) and managed relational databases (RDS).
Development Tools	Git, Docker, Postman	Ensures complete containerized environment parity between development and production environments, along with robust API testing.

Frontend Architecture

Single-page web application and cross-platform mobile views connect to the REST API via HTTPS. Client state management ensures rapid UI updates during order selection and checkout.

Backend & Data Layer

ExpressAPIacts as acontroller handling user authentication, business logic execution, and database queries through an optimized PostgreSQL connection pool.

Part C3 — Cloud Services Comparison & Choice

Comparative Matrix of Cloud Providers & Shared Team Folder

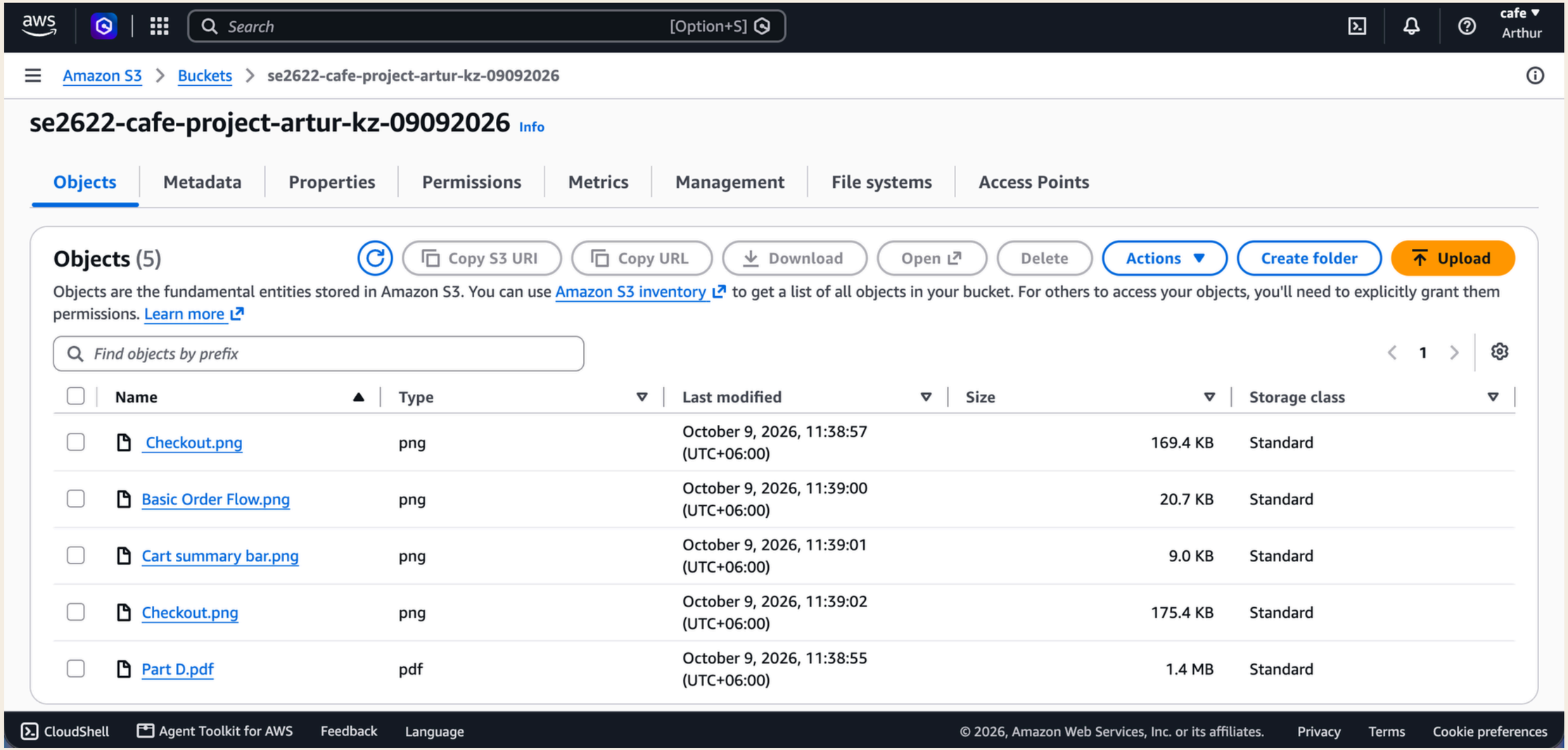
1. CLOUD PROVIDER COMPARISON MATRIX

Criteria	AWS (Selected)	Google Cloud (GCP)	Microsoft Azure
Free Tier Quota	12 Months Free (EC2, RDS) + Always Free Tier	$300 credit for 90 days + free tier limits	$200 credit for 30 days + 12 months free
Collaboration & Access	AWS IAM roles & fine-grained team access	Google Workspace & Cloud IAM integration	Azure Active Directory (Entra ID) integration
Platform Support	Extensive service catalog & container support	Strong Kubernetes (GKE) & container tooling	Native integration with Windows Server & .NET
Security Features	AWS KMS, IAM policies, GuardDuty, MFA	Cloud KMS, Identity-Aware Proxy (IAP)	Azure Key Vault & Defender for Cloud
Post-Free Tier Cost	Pay-as-you-go, flexible reserved options	Sustained use discounts automatically applied	Enterprise agreements & hybrid benefits

Cloud Choice Justification: AWS was selected due to its generous 12-month Free Tier covering both compute (EC2) and managed relational database storage (RDS PostgreSQL). It provides enterprise reliability, seamless scaling, and team IAM access controls for development management.

Part C3 We use Amazon S3 for store our project files. Our team can keep project documents and interface files in cloud.

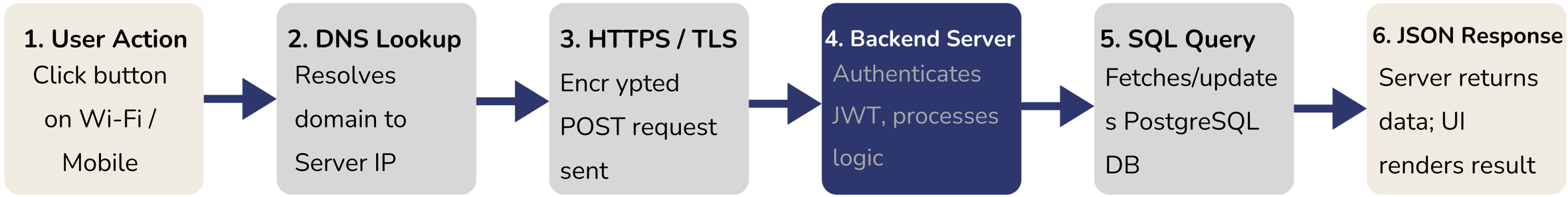
AWS service in use



Part C4 — Data Flow & Security Architecture

End-to-End Request Journey & Protection Mechanisms

1. END-TO-END REQUEST DATA FLOW



2. SECURITY MECHANISMS & PROTECTION CONTROLS

Data in Transit

Enforced **HTTPS / TLS 1.3** encryption prevents eavesdropping and Man-inthe-Middle (MitM) attacks during transmission.

Data at Rest

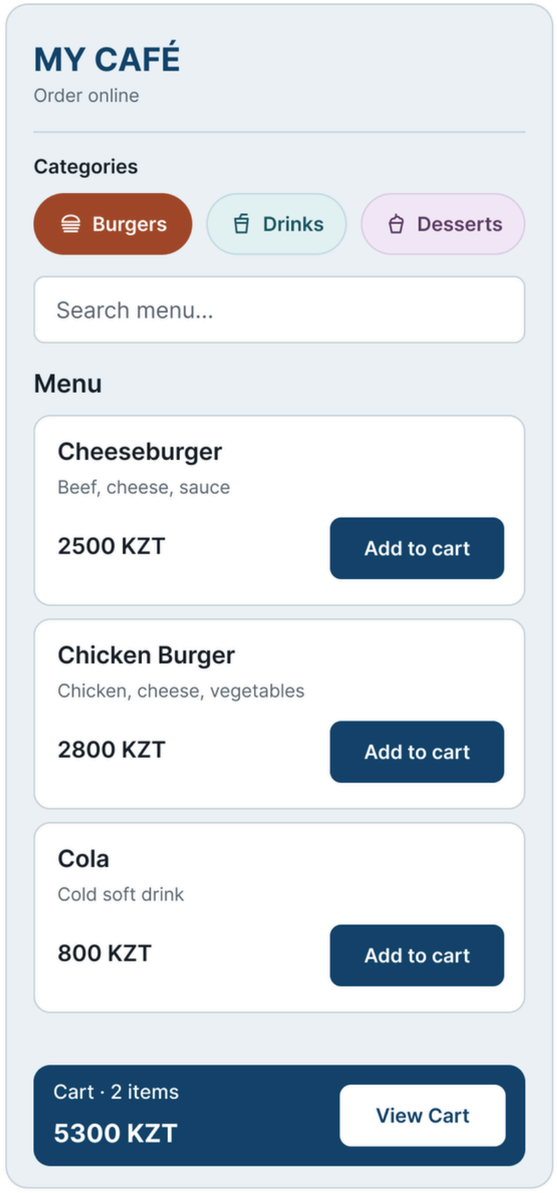
Database storage, customer records, and cloud backups are encrypted using standard **AES-256** algorithms.

Auth & Permissions

Stateless **JWT (JSON Web Tokens)** for session validation paired with **RBAC (Role-Based Access Control).**

Summary of Flow & Security: When a user initiates an order, data payload integrity is secured via HTTPS/TLS 1.3. The Node.js server authenticates requests via JWT, checks user roles, executes parameterized SQL queries against PostgreSQL, and returns an encrypted JSON payload rendered on the client UI.

D1: Customer Menu



MAIN USER TASK

Customer can see café food and drinks, check prices and add items to the cart.

NIELSEN HEURISTIC 1

Recognition rather than recall

Product names, categories and prices are on the screen. Customer does not need to remember them.

NIELSEN HEURISTIC 2

Visibility of system status

The cart shows how many items were added and the total price. Customer can check it at any time.

ACCESSIBILITY DECISION

Buttons are big and text is easy to read. Dark text and light background help people use it on a phone.

Part D

D1 Customer Menu

D2 — Cart / Checkout

‹ **Your Order**

Selected products

Cheeseburger 2500 KZT

Quantity

– 2 + Remove

Subtotal: 5000 KZT

Cola 800 KZT

Quantity

– 1 + Remove

Subtotal: 800 KZT

Subtotal 5800 KZT

Total **5800 KZT**

Pickup

Estimated preparation: 15 min

PLACE ORDER

MAIN USER TASK

The customer can review selected products, change quantities and place the order.

NIELSEN HEURISTIC 1

Error prevention

The customer can review products, quantities and the total price before submitting the order.

NIELSEN HEURISTIC 2

User control and freedom

The customer can change the quantity or remove a product before placing the order.

ACCESSIBILITY DECISION

Clear labels, large controls and readable prices help users avoid mistakes.

D2 Cart / Checkout

D3: Admin Orders

ADMIN PANEL

Orders

Order status

All | New | Preparing | Ready

Order #152 NEW

2 × Cheeseburger · 1 × Cola

Total: 5800 KZT

Accept Order

Order #151 PREPARING

1 × Chicken Burger

Total: 2800 KZT

Mark as Ready

ORDER WORKFLOW

New → Preparing → Ready

Then Completed or Cancelled

Orders | Manage menu

MAIN USER TASK

Café staff can see new orders and change their status.

NIELSEN HEURISTIC 1

Visibility of system status

Each order shows its status. Staff can see New, Preparing or Ready.

NIELSEN HEURISTIC 2

Match between system and the real world

The steps follow how a café works. Orders move from New to Preparing, then Ready and Completed.

ACCESSIBILITY DECISION

Every status has a text label. Buttons are large, so staff can use them fast.

D3 Admin Orders

part D

The menu screen help customer see products prices and categories. Customer can add food to cart and see how many items already added. Cart screen help customer check products change quantity remove food and see total price before order. This can help prevent mistakes. Admin screen help cafe staff see new orders and change order status. We use Nielsen heuristics like visibility of system status recognition rather than recall error prevention and user control. Large buttons clear text and simple labels make interface easier to use on phone and help customer and cafe staff use system faster

Part E

E1

week	task	Responsible person	result
week 6	Finalize UI design, database structure and project setup	Arthur and Adiyat	Final wireframes and database schema
week 7	Develop menu and cart frontend	Arthur	Working menu and cart interface
week 8	Connect backend, database and ordering functions	Arthur and Adiyat	Orders can be created and stored in the database
week 9	Integration, testing and bug fixing	Erdaulet and Adiyat	Working prototype ready for demonstration

E1

END-TERM MILESTONE

IN WEEK 9 WE WILL SHOW WORKING PROTOTYPE OF OUR CAFE SYSTEM. CUSTOMER CAN SEE MENU ADD FOOD TO CART AND MAKE ORDER. AFTER THIS CUSTOMER SEE CONFIRMATION. CAFE STAFF CAN SEE ORDER IN ADMIN PANEL AND CHANGE STATUS

E2
RISKS

RISK	LIKELIHOOD	IMPACT	MITIGATION
Development takes longer than expected	Medium	High	Develop the core ordering functions first
A team member cannot complete work on time	Medium	Medium	Share all project files and redistribute tasks
Cloud service has a limitation or temporary problem	low	Medium	Monitor usage and keep a backup copy of project data